Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mpat functions

Synopsis

The functions prefixed with Mpat make up the Pattern Matching module. The Pattern Matching module is a set of functions that use normalized grayscale correlation (NGC) to search for occurrences of a pattern (model) in an image.

Functions

MpatAlloc
MpatAllocResult
MpatControl
MpatDefine
MpatDraw
MpatFind
MpatFree
MpatGetResult
MpatInquire
MpatMask
MpatPreprocess
MpatRestore
MpatSave
MpatStream

MpatAlloc

Synopsis

Allocate a Pattern Matching context.

Syntax

```
AIL_ID MpatAlloc(
    AIL_ID SysId,           //in
    AIL_INT64 ContextType,  //in
    AIL_INT64 ControlFlag,  //in
    AIL_ID *ContextPatIdPtr //out
)
```

Description

This function allocates a Pattern Matching context on the specified system. A Pattern Matching context contains all the information necessary to perform an MpatFind() operation on a target image, including global search settings and one or more models.

Define search models and add to them to the Pattern Matching context using MpatDefine(). Note that you must add at least one model to the context.

You can adjust the Pattern Matching context settings using MpatControl(). To inquire the Pattern Matching context settings, use MpatInquire().

When the Pattern Matching context is no longer required, release it using MpatFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the Pattern Matching context's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the Pattern Matching context.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, which you have allocated using the MsysAlloc() function.	c		e	f	g	a		i		j	k	l	m	n	b

ContextType

Specifies the type of Pattern Matching context. The parameter must be set to the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the types of Pattern Matching context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_NORMALIZED.	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZED	Specifies a normalized grayscale correlation (NGC) Pattern Matching context.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

ContextPatIdPtr

Specifies the address of the variable in which to write the Pattern Matching context identifier or specifies the data type that the function should use to return the Pattern Matching context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated Pattern Matching context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated Pattern Matching context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_PAT_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the Pattern Matching context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated Pattern Matching context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the Pattern Matching context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_PAT_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailpat11.lib.
DLL	Requires ail11.dll; ailpat11.dll.

MpatAllocResult

Synopsis

Allocate a Pattern Matching result buffer.

Syntax

```
AIL_ID MpatAllocResult(
    AIL_ID SysId,          //in
    AIL_INT64 ControlFlag, //in
    AIL_ID *ResultPatIdPtr //out
)
```

Description

This function allocates a Pattern Matching result buffer, on the specified system, to store results obtained from an MpatFind() operation.

When the Pattern Matching result buffer is no longer required, release it using MpatFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the Pattern Matching result buffer's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the system on which to allocate the result buffer.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ResultPatIdPtr

Specifies the address of the variable in which to write the Pattern Matching result buffer identifier or specifies the data type that the function should use to return the Pattern Matching result buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated Pattern Matching result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated Pattern Matching result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_PAT_ID* is returned instead of a standard Aurora Imaging Library identifier. This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the Pattern Matching result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated Pattern Matching result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the Pattern Matching result buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_PAT_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID

was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailpat11.lib.
DLL	Requires ail11.dll; ailpat11.dll.

MpatControl

Synopsis

Control a setting of a Pattern Matching context, one or all models, or Pattern Matching result buffer.

Syntax

```
void MpatControl(
    AIL_ID ContextOrResultPatId, //out
    AIL_INT Index,               //in
    AIL_INT64 ControlType,       //in
    AIL_DOUBLE ControlValue      //in
)
```

Description

This function allows you to control a setting of a Pattern Matching context, at least one (or all) of the models contained therein, or a Pattern Matching result buffer. These settings control the execution of MpatFind() operations. Most of the control type settings can be inquired using MpatInquire().

Changing certain control type settings might require preprocessing of the Pattern Matching context again, using MpatPreprocess(). To know if the Pattern Matching context needs to be preprocessed, call MpatInquire() with M_PREPROCESSED, after adjusting all required control types.

Any value that you provide to the Aurora Imaging Library Pattern Matching module (for example, position and width) must be specified in the pixel coordinate system, even if the target image is calibrated.

You can change the search region at any time without preprocesing the Pattern Matching context again (for example, when tracking an object, the search region could be changed before each call to MpatFind()).

> Note that when using a C compiler (not a C++ or other compiler) in 64-bit mode, **MpatControl()** internally calls the *AIL_DOUBLE* version of this function (MpatControlDouble()). If you need to pass large integer values (including pointers), call the *AIL_INT64* version of this function (MpatControlInt64()).

Parameters

ContextOrResultPatId

Specifies the Pattern Matching context or Pattern Matching result buffer whose settings you want to modify. The Pattern Matching context or result buffer must have been previously allocated on the system using MpatAlloc() or MpatAllocResult(), respectively.

Index

Specifies to control a Pattern Matching context, an individual model, or a Pattern Matching result buffer. Set this parameter to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a context, or a model index																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Specifies the default. If a Pattern Matching context is specified, same as M_ALL. If a Pattern Matching result buffer is specified, same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Applies the specified control setting to all models, if a Pattern Matching context is specified.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTEXT	Controls a general setting of a specified Pattern Matching context, if one is specified.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Controls a general setting of a Pattern Matching result buffer when ContextOrResultPatId is a result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the individual model to control, if a Pattern Matching context is specified.	c		e	f	g	a		i		j	k	l	m	n	b

ControlType

Specifies the setting to change.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the setting's new value.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- For the general settings of the Pattern Matching context
- For an individual model or all models in a Pattern Matching context
- For controlling advanced settings
- For the result buffer identifier

The following control type and control value allows you specify general settings for a Pattern Matching context. For controlling general context settings, the Index parameter must be set to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the general settings of the Pattern Matching context																
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
M_SEARCH_MODE	Sets the search mode to use when performing an MpatFind() operation. INQ															

	M_DEFAULT	Same as M_FIND_ALL_MODELS.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIND_ALL_MODELS	Specifies to search for all the specified models in the target image using their own search settings. Note that, all models must use the same search region. Each model's search settings also determine the maximum number of occurrences of that model to search for in the target image (M_NUMBER).	c		e	f	g	a		i		j	k	l	m	n	b
	M_FIND_BEST_MODELS	Specifies to search for the best occurrences of any model in the target image, using the search settings from the first model in the Pattern Matching context. Consequently, the search settings of the other models are ignored. Note that, all models must be of the same size. The number of different matches that are found depends on M_NUMBER, M_CERTAINTY, and M_ACCEPTANCE, but cannot exceed 100 occurrences, unless M_MAX_INITIAL_PEAKS is set to M_ALL. You cannot use M_FIND_BEST_MODELS to perform an angular search (M_SEARCH_ANGLE). > Note that, if the first model in the Pattern Matching context is to be ignored (that is, the number of occurrences for that model is set to 0), the settings of the first model whose number of occurrences is not set to 0 will be used instead.	c		e	f	g	a		i		j	k	l	m	n	b

The following control types and control values allow you to control the settings for an individual model or all models in a Pattern Matching context. For controlling the settings of one or all models, the Index parameter must be set to the model index of a specific model in the Pattern Matching context or to M_ALL, respectively.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For an individual model or all models in a Pattern Matching context																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																

M_ACCEPTANCE		Specifies the acceptance level. If the correlation (match score) between the target image and the model is less than this level, it is not considered a match (an occurrence). A perfect match is 100%, whereas no correlation is 0%. The match score depends on the image quality. You should experiment to decide what is a typical match score for your application. Note that the acceptance level must be less than or equal to the certainty level (M_CERTAINTY). INQ														
	0.0 <= *Value* <= 100.0	Specifies the acceptance level, as a percentage. The default value is 70 percent.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCURACY		Specifies the required positional accuracy. You can enhance speed performance by selecting a lower positional accuracy. Note that the search speed slightly affects the positional accuracy. To change the search speed, use M_SPEED. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_HIGH	Specifies a high accuracy (typically ± 0.05 pixels).	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOW	Specifies a low accuracy (typically ± 0.20 pixels). Note that when using this setting, the match scores can be slightly lower than usual. If a precise match score is important to you, use at least medium accuracy.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEDIUM	Specifies a medium accuracy (typically ± 0.10 pixels). This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b

M_CERTAINTY	Specifies the certainty level. If the correlation (match score) between the target image and the model is greater than or equal to the certainty level, a match is assumed without looking elsewhere in the image for a better match. If you set the certainty level too high (close to 100%), you slow down the search because you force the search algorithm to check the whole search region for the best match. A good certainty level is slightly lower than the expected score, so that the search can finish as soon as the match is found. However, if you set the certainty level too low, false matches might be found. Note that the certainty level must be greater than or equal to the acceptance level (M_ACCEPTANCE). INQ	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the certainty level, as a percentage. The default value is 80.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER	Specifies the maximum expected number of occurrences of a model in a target image. This number is used when searching, using MpatFind(). Occurrences with match scores that are greater than or equal to the certainty level (set with M_CERTAINTY)) are returned, up to the number specified by M_NUMBER. If such occurrences are fewer than the specified number, the remaining occurrences returned are the best of those that are greater than or equal to the acceptance level (set with M_ACCEPTANCE). Note that, if a model in the Pattern Matching context is to be ignored, set the number of occurrences for that model (M_NUMBER) to 0. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to look for all matches that are greater than or equal to the acceptance level.	c		e	f	g	a		i		j	k	l	m	n	b

	Value >= 0	Specifies the maximum number of occurrences for which to look in the target image. Only integer values are accepted. The default value is 1.	c		e	f	g	a		i		j	k	l	m	n	b
M_REFERENCE_X		Sets the X-offset of the model's reference position, relative to the model origin. Position results return the X-coordinate of the model's reference position transformed at the model occurrence. Position results are relative to the target origin. If you redefine the model's reference position (with M_REFERENCE_...), make sure that the search region (defined by M_SEARCH_OFFSET_... and M_SEARCH_SIZE_... or the target image's ROI) covers this new reference position and takes into account the angular search range of the model (specified using M_SEARCH_ANGLE...). Note that although the reference position need not be in the model, it should be inside the search region. If, in the target image, the reference position falls outside target image, Aurora Imaging Library returns a negative position. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-offset of the model's reference position relative to the model origin, in pixels. The default value is (M_ALLOC_SIZE_X-1)/2.	c		e	f	g	a		i		j	k	l	m	n	b

M_REFERENCE_Y	Sets the Y-offset of the model's reference position, relative to the model origin. Position results return the Y-coordinate of the model's reference position transformed at the model occurrence. Position results are relative to the target origin. If you redefine the model's reference position (with M_REFERENCE_...), make sure that the search region (defined by the M_SEARCH_OFFSET_... and M_SEARCH_SIZE_... or the target image's ROI) covers this new reference position and takes into account the angular search range of the model (specified using M_SEARCH_ANGLE...). Note that although the reference position need not be in the model, it should be inside the search region. If, in the target image, the reference position falls outside target image, Aurora Imaging Library returns a negative position. INQ	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the Y-offset of the model's reference position relative to the model origin, in pixels. The default value is (M_ALLOC_SIZE_Y-1)/2.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATED_MODEL_MINIMUM_SCORE	Sets whether to use angle refinement mode, as well as sets the score to use for this mode. During preprocessing, this control type helps establish the full range of degrees within which a rotated version of the model can be rotated from the model at a specific angle and still match with at least the score specified by this control type (the angle refinement score). The higher the angle refinement score is set, the smaller the angle refinement step will be. The lowest value between the determined angle refinement step and the specified angle accuracy (set using M_SEARCH_ANGLE_ACCURACY) is the refinement angle used when searching for the pattern in the target image. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies to disable the angle refinement mode.	c		e	f	g	a		i		j	k	l	m	n	b

	0.0 <= *Value* <= 100.0	Specifies the minimum score.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE		Sets the nominal search angle at which to search for occurrences of the model. For best results, if there is a region of interest (ROI) in your target image, set this control's value to 0.0. Note that there will be an increase in processing time if the model is at an angle. Angles are interpreted with respect to the pixel coordinate system, so they are always measured counter-clockwise. MpatFind() does not support angular searches when M_SEARCH_MODE is set to M_FIND_BEST_MODELS. When M_SEARCH_ANGLE_MODE is disabled, this setting has no effect. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 0.0.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the nominal search angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_ACCURACY		Sets the required precision with which to find the angle of the occurrence. The precision defines an angular range in which to fine-tune the search after the approximate location of the occurrence of the model is found. To be effective, you must set the degree of accuracy to a value smaller than that of the rotation tolerance (M_SEARCH_ANGLE_TOLERANCE). This angular range is further restricted by M_ROTATED_MODEL_MINIMUM_SCORE. For more information, refer to the Search constraints section of Chapter 7: Pattern matching. The lower the accuracy, the higher the precision of the occurrence angle. MpatFind() does not support angular searches when M_SEARCH_MODE is set to M_FIND_BEST_MODELS. When M_SEARCH_ANGLE_MODE is disabled, this setting has no effect. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies that the angle accuracy equals the tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 < *Value* <= 180.0	Specifies the angle accuracy, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_DELTA_NEG		Sets the lower limit for the angular range, relative to the nominal search angle (set using M_SEARCH_ANGLE). The angular range should be used to cover the expected variance in angle. Occurrences with angles outside the angular-range, +/- a tolerance, cannot be returned as results. If M_SEARCH_ANGLE_ACCURACY is set to M_DISABLE, the tolerance is equal to M_SEARCH_ANGLE_TOLERANCE. Otherwise, the tolerance is equal to 1.5 * M_SEARCH_ANGLE_TOLERANCE. Note that the greater the range of angles to be searched, the slower the search. To set the upper limit for the angular range, use M_SEARCH_ANGLE_DELTA_POS. MpatFind() does not support angular searches when M_SEARCH_MODE is set to M_FIND_BEST_MODELS. When M_SEARCH_ANGLE_MODE is disabled, this setting has no effect. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value. The default value is 180.0 when the model is of type M_CIRCULAR_OVERSCAN, and 0.0 otherwise.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 180.0	Specifies the angle in a clockwise rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b

M_SEARCH_ANGLE_DELTA_POS	Sets the upper limit for the angular range, relative to the nominal search angle (set using M_SEARCH_ANGLE). The angular range should be used to cover the expected variance in angle. Occurrences with angles outside the angular-range, +/- a tolerance, cannot be returned as results. If M_SEARCH_ANGLE_ACCURACY is set to M_DISABLE, the tolerance is equal to M_SEARCH_ANGLE_TOLERANCE. Otherwise, the tolerance is equal to 1.5 * M_SEARCH_ANGLE_TOLERANCE. Note that the greater the range of angles to be searched, the slower the search. To set the lower limit for the angular range, use M_SEARCH_ANGLE_DELTA_NEG. MpatFind() does not support angular searches when M_SEARCH_MODE is set to M_FIND_BEST_MODELS. When M_SEARCH_ANGLE_MODE is disabled, this setting has no effect. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value. The default value is 180.0 when the model is of type M_CIRCULAR_OVERSCAN, and 0.0 otherwise.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 180.0	Specifies the angle in a counter-clockwise rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_INTERPOLATION_MODE	Sets the type of interpolation to use when rotating the model to perform the search at an angle. MpatFind() does not support angular searches when M_SEARCH_MODE is set to M_FIND_BEST_MODELS. When M_SEARCH_ANGLE_MODE is disabled, this setting has no effect. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_NEAREST_NEIGHBOR.	c		e	f	g	a		i		j	k	l	m	n	b

	M_BICUBIC	Specifies bicubic interpolation. The new value is determined by taking a weighted average of the 16 values (4x4) that surround the source point. Note that the sum of the weights used for bicubic interpolation might be greater than one. If this occurs and the result reflects an overflow or underflow, the result is saturated.	c		e	f	g	a		i		j	k	l	m	n	b
	M_BILINEAR	Specifies bilinear interpolation. The new value is determined by taking a weighted average of the 4 values (2x2) that surround the source point.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. The new value is that of the pixel closest to the source point.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_MODE		Sets whether to perform calculations specific to angular-range search strategies. To search for models at an angle (within their specified angular range M_SEARCH_ANGLE-M_SEARCH_ANGLE_DELTA_NEG to M_SEARCH_ANGLE + M_SEARCH_ANGLE_DELTA_POS in the target) this control type must be enabled. In addition, you can limit the number of angles searched using M_SEARCH_ANGLE_ACCURACY and M_SEARCH_ANGLE_TOLERANCE. MpatFind() does not support angular searches when M_SEARCH_MODE is set to M_FIND_BEST_MODELS. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable the angular search mode. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable the angular search mode.	c		e	f	g	a		i		j	k	l	m	n	b

M_SEARCH_ANGLE_TOLERANCE	Sets the full range of degrees within which the pattern in the target image can be rotated from a model at a specific angle and still meet the acceptance level. For example, if a model can tolerate the target image being offset from its search angle by ±2.5°, specify 5°. The specified tolerance determines the step angle. Note that very small tolerance values should only be used within small angular ranges; otherwise, application performance can be adversely affected. For more information, refer to the Finding models when they are at an angle section of Chapter 7: Pattern matching. MpatFind() does not support angular searches when M_SEARCH_MODE is set to M_FIND_BEST_MODELS. When M_SEARCH_ANGLE_MODE is disabled, this setting has no effect. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 5.0 degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_AUTO	Specifies that the tolerance will be automatically determined based on an analysis of the model.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 < *Value* <= 180.0	Specifies the step angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_OFFSET_X	Specifies the X-offset of the specified model's search region. It limits the area in the target image in which to find the reference position of the model (set with M_REFERENCE_...), and consequently increases the search speed. Note that, if searching for multiple models and your M_SEARCH_MODE is set to M_FIND_ALL_MODELS, the offset of the search region must be the same for all the models. The search region can fall outside the target area, specified when calling MpatFind(). When dealing with a target image with an ROI, this value is ignored. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Specifies the X-coordinate of the top-left corner of the target image. By default, the entire target image is searched.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-coordinate, relative to the pixel coordinate system, of where to start the search. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_OFFSET_Y		Specifies the Y-offset of the specified model's search region. It limits the area in the target image in which to find the reference position of the model (set with M_REFERENCE_...), and consequently increases the search speed. Note that, if searching for multiple models and your M_SEARCH_MODE is set to M_FIND_ALL_MODELS, the offset of the search region must be the same for all the models. The search region can fall outside the target area, specified when calling MpatFind(). When dealing with a target image with an ROI, this value is ignored. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the Y-coordinate of the top-left corner of the target image. By default, the entire target image is searched.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-coordinate, relative to the pixel coordinate system, of where to start the search. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b

M_SEARCH_SIZE_X		Specifies the width of the search region. Note that the size of the search region can be smaller than the size of the model because the search region only defines where to find the reference position. Note that, if searching for multiple models and your M_SEARCH_MODE is set to M_FIND_ALL_MODELS, the width of the search region must be the same for all the models. The search region can fall outside the target image, specified when calling MpatFind(). When dealing with a target image with an ROI, this value is ignored. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies to use the width of the full image.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the width, in pixels. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_SIZE_Y		Specifies the height of the search region. Note that the size of the search region can be smaller than the size of the model because the search region only defines where to find the reference position. Note that, if searching for multiple models and your M_SEARCH_MODE is set to M_FIND_ALL_MODELS, the height of the search region must be the same for all the models. The search region can fall outside the target image, specified when calling MpatFind(). When dealing with a target image with an ROI, this value is ignored. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies to use the height of the full image.	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the height, in pixels. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_SPEED		Specifies the required search speed when using MpatFind(). At a higher speed, the search takes all reasonable short cuts; therefore, the search should be performed faster than at lower settings. Generally, the high speed setting should be used for better quality images or when using a simple model. Note, the high speed setting reduces positional accuracy very slightly. Try the low speed settings only if your image quality is particularly poor and you have encountered problems at higher speeds. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_HIGH	Specifies a high search speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOW	Specifies a low search speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEDIUM	Specifies a medium search speed. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_VERY_HIGH	Specifies a very high search speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_VERY_LOW	Specifies a very low search speed.	c		e	f	g	a		i		j	k	l	m	n	b

The following control types and control values are for advanced users of the Pattern Matching module. If the default settings do not satisfy the requirements of your application, these control types and control values can be used to fine tune the model's search settings. The default settings typically provide the best results for most search operations. To gain a clearer understanding of how the search algorithm works, see the Pattern matching algorithm (for advanced users) section of Chapter 7: Pattern matching.

For these control types, the Index parameter must be set to the model index of a specific model in the Pattern Matching context or M_ALL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling advanced settings																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																

M_COARSE_SEARCH_ACCEPTANCE		Sets the coarse search acceptance threshold level to use for rejecting candidate model peaks at low resolution levels. Note that this value is not saved with the Pattern Matching context. Instead, this value must be set each time the Pattern Matching context is restored. Changing the coarse search acceptance to a specified value might increase the search time when some of the matches you request do not reach the certainty level, or when you request more matches than are really present in the image. The coarse search acceptance level should usually be set much lower than the search acceptance level. For example, a good level to set it to is about 20% to 30%. Note that if it is too low, you will not see any increase in speed. However, if it is too high, you risk rejecting real match peaks. Note that the coarse search acceptance threshold level is not saved and restored with the Pattern Matching context. So, if you always want to use your own coarse search acceptance level, you should set it each time after restoring your Pattern Matching context and before calling MpatFind(). Otherwise, the default coarse search acceptance level is used. INQ														
	M_DEFAULT	Specifies to determine the coarse search acceptance threshold level automatically.	c		e	f	g	a		i		j	k	l	m	n	b
	1.0 <= *Value* <= 100.0	Specifies the coarse search acceptance threshold level.	c		e	f	g	a		i		j	k	l	m	n	b

M_EXTRA_CANDIDATES	Sets the number of extra candidates to consider. Normally, the search algorithm considers only a limited number of (best) scores as possible candidates to match when proceeding at the most subsampled stage. This setting allows you to add robustness to the algorithm, by considering more candidates, without overly compromising the search speed. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of extra candidates. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_FAST_FIND	Sets whether to use fast peak finding. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies to use preprocessing to decide if fast peak finding is appropriate.	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies not to use fast peak finding. The initial search (at the resolution level determined by M_FIRST_LEVEL) computes the correlation at every position in the search region. This guarantees that the biggest match peak will be found, and that it will be investigated first.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies to use fast peak finding. The search algorithm attempts to find the peaks without checking every point. This is safe in most cases, but can cause matches to be missed for models that produce very narrow peaks.	c		e	f	g	a		i		j	k	l	m	n	b

M_FIRST_LEVEL		Sets the resolution level for the initial stage (lowest resolution) of the search. Note that level 0 is the original target image and each higher level is half the size (and resolution) of the previous one. If the specified level is not supported by the search algorithm, the highest possible level will be used. A higher first level speeds up the initial search but makes it less reliable, because the model might not retain enough distinctive features at such a low resolution. After preprocessing your Pattern Matching context, use MpatInquire() with M_PROC_FIRST_LEVEL to inquire about the default value of the lowest resolution level. Note that if you set a lower value for M_FIRST_LEVEL than the one assigned to M_LAST_LEVEL, **MpatControl**() will override your settings and automatically assign M_LAST_LEVEL the same value as M_FIRST_LEVEL. This implies that only one resolution level will be used for the search. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_AUTO_SIZE_BASED.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO_CONTENT_BASED	Specifies to automatically determine the first level based on an analysis of the model contents.	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO_SIZE_BASED	Specifies to automatically determine the first level based on the model size.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= *MaxLevel*	Specifies the first level, where MaxLevel is the maximum possible level. After preprocessing your Pattern Matching context, use MpatInquire() with M_MODEL_MAX_LEVEL to establish the maximum possible level. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b

M_LAST_LEVEL	Sets the resolution level for the final stage (highest resolution) of the search. Note that if the specified level is not supported by the search algorithm, the highest acceptable level will be used. Search score can also be less reliable for levels above 0, depending on the characteristics of the model. After preprocessing your Pattern Matching context, use MpatInquire() with M_PROC_LAST_LEVEL to inquire about the default value of the highest resolution level. Note that if you set a higher value for M_LAST_LEVEL than the one assigned to M_FIRST_LEVEL, **MpatControl**() will override your settings and automatically assign M_LAST_LEVEL the same value as M_FIRST_LEVEL. This implies that only one resolution level will be used for the search. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the last level automatically.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= *MaxLevel*	Specifies the last level, where MaxLevel is the maximum possible level. After preprocessing your Pattern Matching context, use MpatInquire() with M_MODEL_MAX_LEVEL to establish the maximum possible level. Only integer values are accepted.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_INITIAL_PEAKS	Sets the maximum number of returned peaks after an initial search at the first level for occurrences of multiple models, or a single model with circular overscan data (that is, M_AUTO_MODEL or M_REGULAR_MODEL with M_CIRCULAR_OVERSCAN). INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies to limit the returned number of peaks for optimal performance.	c		e	f	g	a		i		j	k	l	m	n	b

	M_ALL	Specifies all occurrences that pass the model acceptance threshold should be returned. Performance might be reduced. It is recommended to use this setting when the target image has many potential false positive occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_SEPARATION_X		Sets the minimum separation (in X) between two occurrences of the same model in order for them to be considered distinct. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Sets the minimum separation to 75% of the model width (Param3 of MpatDefine()) when creating an M_REGULAR_MODEL.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the minimum separation as a percentage of the model's width. This value can be greater than 100%.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_SEPARATION_Y		Sets the minimum separation (in Y) between two occurrences of the same model in order for them to be considered distinct. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Sets the minimum separation to 75% of the model height (Param4 of MpatDefine()) when creating an M_REGULAR_MODEL).	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the minimum separation as a percentage of the model's height. This value can be greater than 100%.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODEL_STEP		Sets whether all or every second pixel in the model is used in the correlation during the high resolution stage of the search. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies to automatically establish the model step.	c		e	f	g	a		i		j	k	l	m	n	b

	1	Specifies to use all model pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	2	Specifies to use every second model pixel (in both the X and Y directions). This speeds up the last (high resolution) stage of the search, particularly for large models. The match score can be affected if the model has many fine features, but will tend not to be affected if the model has mainly coarse features.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_ACCURACY_MODE		Sets the method for scanning the specified angular range for multiple occurrences, or a single occurrence with circular overscan data (that is, M_AUTO_MODEL or M_REGULAR_MODEL with M_CIRCULAR_OVERSCAN). INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies to return the closest occurrence to the approximate location within the specified angular range, for optimal performance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies that all possible occurrences that are found within the range of the accuracy specified with M_SEARCH_ANGLE_ACCURACY will be scanned, and the best will be returned. Performance might be reduced. It is recommended to use this setting when the target image has many potential false positive occurrences.	c		e	f	g	a		i		j	k	l	m	n	b

The following control types and control values settings can be specified for a Pattern Matching result buffer. For controlling the settings of a Pattern Matching result buffer, the Index parameter must be set to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the result buffer identifier																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																

M_RESULT_OUTPUT_UNITS	Sets whether to return results in pixels or world units. This essentially sets the output coordinate system to use. The setting of this control type will only affect functions within this module which return positional results. This control type can be changed at any time to return results in the required output units. INQ															
M_DEFAULT	Same as M_ACCORDING_TO_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, it specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. If world units are specified, calling MpatGetResult() generates an error if the result was not calculated on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE_SUMS	Sets whether to save the values of the sums used to compute the normalized correlation result for each model occurrence. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the sums are not saved. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the sums are saved.	c		e	f	g	a		i		j	k	l	m	n	b
M_TARGET_CACHING	Sets whether the pyramidal representation of the target buffer is kept in the result buffer or generated each time MpatFind() is called. INQ	c		e	f	g	a		i		j	k	l	m	n	b

	M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the pyramidal representation of the target buffer is generated each time MpatFind() is called.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the pyramidal representation of the target buffer is kept in the result buffer. Note that the pyramidal representation is generated upon the first call to MpatFind(). This pyramidal representation is re-used by consecutive calls to MpatFind() as long as the same result buffer is used and the image, search region, and model size are not modified.	c		e	f	g	a		i		j	k	l	m	n	b

Type-specific versions of the function when using a C compiler under 64-bit

```
void MpatControlInt64 (AIL_ID ContextOrResultPatId, AIL_INT Index, AIL_INT64 ControlType, AIL_INT64
ControlValue)
```

Details

Parameters

ContextOrResultPatId

See ContextOrResultPatId of the main function for a description.

Index

See Index of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

```
void MpatControlDouble (AIL_ID ContextOrResultPatId, AIL_INT Index, AIL_INT64 ControlType, AIL_DOUBLE ControlValue)
```

Details

Parameters

ContextOrResultPatId

See ContextOrResultPatId of the main function for a description.

Index

See Index of the main function for a description.

ControlType

See ControlType of the main function for a description.

ControlValue

See ControlValue of the main function for a description.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailpat11.lib.
DLL	Requires ail11.dll; ailpat11.dll.

MpatDefine

Synopsis

Add one or more models to, or delete a model from, a Pattern Matching context.

Syntax

```
void MpatDefine(
    AIL_ID ContextPatId,  //out
    AIL_INT64 ModelType,  //in
    AIL_ID SrcImageBufId, //in
    AIL_DOUBLE Param1,    //in
    AIL_DOUBLE Param2,    //in
    AIL_DOUBLE Param3,    //in
    AIL_DOUBLE Param4,    //in
    AIL_INT64 ControlFlag //in
)
```

Description

This function allows you to add one or more models at a time to, or delete a model from, a Pattern Matching context. When defining a search model, you are specifying the area of an image to use as a pattern to find within a target image, using MpatFind(). When a call to MpatFind() is made, all models that are part of the specified context (and that have MpatControl() with M_NUMBER set to a value greater than zero) are searched for simultaneously in the target. Note, when a model is added or deleted, the Pattern Matching context must be preprocessed again, using MpatPreprocess().

The search is performed according to the general search settings specified in the Pattern Matching context, as well as the individual model search settings (depending on the M_SEARCH_MODE value specified). Specify both the context and individual model search settings using MpatControl().

When adding a model to the context, a number is assigned to the new model. The context starts numbering the models at 0, and each subsequent model added to the Pattern Matching context is given a sequential index number, in the order that you add the models. When deleting a model, all models with a higher index shift down one.

Parameters

ContextPatId

Specifies the Pattern Matching context to which to add, or from which to delete, the model. The Pattern Matching context must have been previously allocated on the required system using MpatAlloc().

ModelType

Specifies the type of the model to define when adding models to the context, or specifies to delete a model from the Pattern Matching context.

See the Parameter associations section for possible values that can be specified.

SrcImageBufId

Specifies the identifier of the image buffer from which to define the models, or the Pattern Matching context from which to copy a model.

Set this parameter to **M_NULL** if not used.

See the Parameter associations section for possible values that can be specified.

Param1

Specifies an attribute of the model to add. Its definition is dependent on the model type chosen. When not used, set this parameter to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

Param2

Specifies an attribute of the model to add. Its definition is dependent on the model type chosen. When not used, set this parameter to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

Param3

Specifies an attribute of the model to add. Its definition is dependent on the model type chosen. When not used, set this parameter to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

Param4

Specifies an attribute of the model to add. Its definition is dependent on the model type chosen. When not used, set this parameter to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Specifies the speed of the allocation or the interpolation mode to use, depending on the model type chosen. When not used, set this parameter to **M_DEFAULT**.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ModelType**, **SrcImageBufId**, **Param1**, **Param2**, **Param3**, **Param4**, and **ControlFlag** parameters.

- For defining models to add to specified Pattern Matching context
- For deleting the specified model from the Pattern Matching context

To add a model to the Pattern Matching context, the ModelType, Param1, Param2, Param3, and ControlFlag parameters can be set to the following values:

Note that any unused parameters should be set to **M_DEFAULT**. Note, if unused, set SrcImageBufId to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For defining models to add to specified Pattern Matching context																
ModelType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	SrcImageBufId																
	Param1																
	Param2																
	Param3																
	Param4																
	ControlFlag																
M_AUTO_MODEL +		Defines one or more models automatically of the specified dimensions from the most-suitable distinct areas found, in the source image. If none are found, no model is allocated and an error is reported. This is useful when you want to perform whole image alignment, for which allocation of a distinct model is essential. It can take several seconds to find the best models (more for large or small images). To be effective, the source image should be a typical target image. You can only define an M_AUTO_MODEL that respect the following condition: ($maxvalue^2$ * Param3 * Param4) < 2^{63} where *maxvalue* is the maximum pixel value (typically, 255) in the target image and the model. This restriction is imposed to avoid overflows in the internal 64-bit accumulators. The total area of the model must be greater or equal to 4 pixels (Param3 * Param4 >= 4).															
	SrcImageBufId	Sets the identifier of the image buffer (source image) from which to extract a distinct model. The image buffer must be a 1-band, 8-bit, unsigned processing buffer. This image buffer must not have a region of interest (ROI) associated with it; using an image buffer with an ROI will cause an error.	c		e	f	g	a		i		j	k	l	m	n	b
	Param1	Sets the maximum displacement (shift), in the horizontal direction, expected between the position of the model in the source image and the position of the model when found in the target image. This information is used to select models that are far enough from the image borders to be present in the target images.	c		e	f	g	a		i		j	k	l	m	n	b

		M_DEFAULT	Selects models from anywhere in the whole width of the image.	c		e	f	g	a		i		j	k	l	m	n	b
		Value >= 0	Specifies the expected maximum pixel displacement in the horizontal direction, as an *integer*.	c		e	f	g	a		i		j	k	l	m	n	b
	Param2		Sets the maximum displacement (shift), in the vertical direction, expected between the position of the model in the source image and the position of the model when found in the target image. This information is used to select models that are far enough from the image borders to be present in the target images.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Selects models from anywhere in the whole height of the image.	c		e	f	g	a		i		j	k	l	m	n	b
		Value >= 0	Specifies the expected maximum pixel displacement in the vertical direction, as an *integer*.	c		e	f	g	a		i		j	k	l	m	n	b
	Param3		Sets the width of the model, as an *integer*. Note that the width of the model must be equal to or smaller than the width of your source image.	c		e	f	g	a		i		j	k	l	m	n	b
	Param4		Sets the height of the model, as an *integer*. Note that the height of the model must be equal to or smaller than the height of your source image.	c		e	f	g	a		i		j	k	l	m	n	b
	ControlFlag		Sets the speed of the process by which model(s) are defined from the model source image.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Same as M_FAST.	c		e	f	g	a		i		j	k	l	m	n	b
		M_BEST +	Specifies that the search should favor high precision over speed. You should typically use this setting if the model source image is small.	c		e	f	g	a		i		j	k	l	m	n	b
		M_FAST +	Specifies that the search should favor speed over high precision. You should typically use this setting if your model source image is large.	c		e	f	g	a		i		j	k	l	m	n	b

M_REGULAR_MODEL +	Defines a regular model, from the specified area, in the source image. You can use this type of model when dealing with images containing a mix of models on an inconsistent background, such as an image of loose nuts and bolts lying on a metal sheet. If the occurrence can appear at an angle, specify the angular range in which it can appear, using MpatControl() with M_SEARCH_ANGLE.... If the occurrence is expected to be found always with the same background, use the M_CIRCULAR_OVERSCAN combination constant. To extract the model from a rotated region in the model's source image, pass a source image that is associated with a rectangular region of interest (ROI) at the required angle. You can associate an ROI to your source image using MbufSetRegion(). You can only define an M_REGULAR_MODEL model that respects the following condition: $(maxvalue^{2} * Param3 * Param4) < 2^{63}$, where *maxvalue* is the maximum pixel value (typically, 255) in the target image and the model. This restriction is imposed to avoid overflows in the internal 64-bit accumulators. The total area of the model must be greater or equal to 4 pixels (Param3 * Param4 >= 4).	c		e	f	g	a		i		j	k	l	m	n	b
SrcImageBufId	Sets the identifier of the image buffer (model source image) from which to extract the model. The image buffer must be a 1-band, 8-bit, unsigned processing buffer. This image buffer can have a rotated rectangular region of interest (ROI) associated with it, defined using MbufSetRegion() with a 2D graphics list and M_FILL_REGION, and either M_RASTERIZE or M_NO_RASTERIZE. An error will be generated if the ROI has a non-rectangular shape. When specifying an image buffer (model source image) that is associated with an ROI, you must set Param1, Param2, Param3, and Param4 to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b

Param1	Specifies the X-coordinate of the top-left corner of the model (origin) in the model's source image (SrcImageBufId). Note that this X-coordinate must not be outside the model's source image. When specifying a model's source image that is associated with an ROI, you must set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that the X-coordinate of the model origin is 0. Alternatively, if the model's source image is associated with an ROI, it specifies to ignore this parameter.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= *ImageSizeX*	Specifies the X-coordinate of model origin in the source image, in pixels (as an *integer*).	c		e	f	g	a		i		j	k	l	m	n	b
Param2	Specifies the Y-coordinate of the top-left corner of the model (origin) in the model's source image (SrcImageBufId). Note that this Y-coordinate must not be outside the model's source image. When specifying a model's source image that is associated with an ROI, you must set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that the Y-coordinate of the model origin is 0. Alternatively, if the model's source image is associated with an ROI, it specifies to ignore this parameter.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= *ImageSizeY*	Specifies the Y-coordinate of model origin in the source image, in pixels (as an *integer*).	c		e	f	g	a		i		j	k	l	m	n	b
Param3	Specifies the width of the model in the model's source image. Note that the region from which the model is defined must not be outside the model source image. When specifying a model's source image that is associated to an ROI, you must set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b

		M_DEFAULT	Specifies that the width of the model is the distance from the origin of the model to the edge of the image in the X-direction. Alternatively, if the model's source image is associated with an ROI, it specifies to ignore this parameter.	c		e	f	g	a		i		j	k	l	m	n	b
		1 <= *Value* <= *ImageSizeX*	Specifies the width of the model in the model source image, in pixels (as an *integer*).	c		e	f	g	a		i		j	k	l	m	n	b
	Param4		Specifies the height of the model in the model's source image. Note that the region from which the model is defined must not be outside the model's source image. When specifying a model's source image that is associated with an ROI, you must set this parameter to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
		M_DEFAULT	Specifies that the height of the model is the distance from the origin of the model to the edge of the image in the Y-direction. Alternatively, if the model's source image is associated with an ROI, it specifies to ignore this parameter.	c		e	f	g	a		i		j	k	l	m	n	b
		1 <= *Value* <= *ImageSizeY*	Specifies the height of the model in the model source image, in pixels (as an *integer*).	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For defining models to add to specified Pattern Matching context.

You can add the following value to the above-mentioned values to extract the model as well as a circular area around it.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For extracting the circular overscan data																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

		c		e	f	g	a		i		j	k	l	m	n	b
M_CIRCULAR_OVERSCAN	Specifies that the model is extracted from the source image with the circular overscan data. The circular overscan data is determined by rotating the model, about its center. The circular overscan data of an M_CIRCULAR_OVERSCAN type model is used if you search for an occurrence of the model at an angle. Angular search is fastest when performed with an M_CIRCULAR_OVERSCAN model; however, M_CIRCULAR_OVERSCAN should only be used when the region around the model is consistent, for example, when searching for a chip in the image of an integrated circuit. When preprocessing an M_CIRCULAR_OVERSCAN model for which an angular search range has been specified, a set of models is extracted from rotated versions of the M_CIRCULAR_OVERSCAN model, defining models that would appear upright if the target image were rotated. For this type of model, a larger region than the one defined is extracted from the source image so as to allow creation of models at different angles. The model must not be extracted from a region too close to the edge of the source image. In addition, the M_CIRCULAR_OVERSCAN model should be complex enough so that if models are created from it at the required angles, they are representative of the pattern being sought. For a model defined to include circular overscan data, you cannot define "don't care" regions (MpatMask()).															

Combination value for [when ModelType=M_AUTO_MODEL: M_BEST; M_FAST].

You can add the following value to the above-mentioned values to set the number of models to allocate from the same image.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For use with the auto model context																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MULTIPLE + *n*	Allocates several models from the same source image. Set *n* to the number of models to define. For example, set ControlFlag to M_BEST + M_MULTIPLE + 4 to define the four most distinct models available in your model source image. Search models can overlap within the image by half the size of the model (in X and Y).	c		e	f	g	a		i		j	k	l	m	n	b

When deleting a model from the specified Pattern Matching context, use the following value:

Any unused parameters should be set to **M_DEFAULT**. Note, if unused, set SrcImageBufId to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For deleting the specified model from the Pattern Matching context																	
ModelType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
SrcImageBufId																
Param1																
Param2																
Param3																
Param4																
ControlFlag																
M_DELETE	Specifies to remove the specified model from the Pattern Matching context, specified using the Param1 parameter. When deleting a model, all models with a higher index shift down one.															
SrcImageBufId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Param1	Specifies the model to delete from within the Pattern Matching context.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to delete all models from the Pattern Matching context.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the model to delete.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailpat11.lib.
DLL	Requires ail11.dll; ailpat11.dll.

MpatDraw

Synopsis

Draw specific features of the model or result occurrence in an image buffer or 2D graphics list.

Syntax

```
void MpatDraw(
    AIL_ID ContextGraId,           //in
    AIL_ID ContextOrResultPatId,   //in
    AIL_ID DstImageBufOrListGraId, //out
    AIL_INT64 Operation,           //in
    AIL_INT Index,                 //in
    AIL_INT64 ControlFlag          //in
)
```

Description

This function draws specific model or result occurrence features in the destination image buffer or 2D graphics list. **MpatDraw**() can be used with multiple results.

Parameters

ContextGraId

Specifies the identifier of the 2D graphics context to use. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 2D graphics context																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the default 2D graphics context of the current Aurora Imaging Library application is used. > Note that there is a different default 2D graphics context for each thread.	c		e	f	g	a		i		j	k	l	m	n	b
2D graphics context identifier	Specifies a valid 2D graphics context identifier, which you have allocated using MgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextOrResultPatId

Specifies the Pattern Matching context or result buffer identifier from which to extract the features to draw. The Pattern Matching context or result buffer must have been previously allocated using MpatAlloc() or MpatAllocResult(), respectively.

DstImageBufOrListGraId

Specifies the identifier of the destination image buffer or 2D graphics list into which to draw. The buffer

can be any valid 1- or 3-band image buffer allocated using MbufAlloc...(). The 2D graphics list must be previously allocated using MgraAllocList(). By drawing into a display's overlay buffer or associating the 2D graphics list with the display, you can also annotate an image non-destructively.

If specifying an image buffer, it must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

When using **MpatDraw**() to draw features extracted from models or targets that are associated with a camera calibration context, the destination drawing buffer must also be calibrated to the same world coordinate system. **MpatDraw**() draws all the requested features according to whether the destination image buffer is physically corrected or not.

When drawing a model, the destination image must be at least the size of the model.

Operation

Specifies the type of operation to perform. Operations can be added together to draw multiple features at a time. For example, to draw both the result occurrence's bounding box and position, you would specify M_DRAW_BOX + M_DRAW_POSITION as the Operation parameter. The Operation parameter can be set to one or a combination of the values below.

The following operations can extract the features to draw from both a model and a result buffer.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying an operation that can extract features from both a model and a result buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_BOX +	Draws the bounding box of the model or of the result occurrence(s). For a result occurrence, the box is drawn at the coordinates at which its origin was found in the target image (not the reference position), respecting the occurrence's angle.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_POSITION +	Draws a cross at the model's reference position, or at the result occurrence(s) position and angle found in the target image (with respect to the reference position).	c		e	f	g	a		i		j	k	l	m	n	b

Combination values that can be used alone or as combination values for the values listed in For specifying an operation that can extract features from both a model and a result buffer.

You can use one or more of the following values in combination with each other or with the above-mentioned values, to specify an operation that can extract features to draw only from a model.

The following operations can extract the features to draw only from a model and cannot draw in a 2D graphics list.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying an operation that can extract features to draw only from a model																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DRAW_DONT_CARE	Draws the model's don't care pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_IMAGE	Draws the region of the model source image from which the model was extracted.	c		e	f	g	a		i		j	k	l	m	n	b

Index

Specifies whether to draw a specific result occurrence or model, or to draw all of them.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the index																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value. If a Pattern Matching context is specified, the default is 0. If a Pattern Matching result buffer is provided, the default is the same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Draws the specified feature of all models or all result occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the model index or the result occurrence.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Specifies where to draw a model. This parameter must be set to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying where to draw the model																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies to draw the model starting from the center of the top-left pixel of the destination image; or, if drawing in a 2D graphics list, from the center of the top-left pixel of the image used at the time of annotation.	c		e	f	g	a		i		j	k	l	m	n	b
M_ORIGINAL	Specifies to draw the model starting from the offsets used to define the model in the model source image.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailpat11.lib.
DLL	Requires ail11.dll; ailpat11.dll.

MpatFind

Synopsis

Find a pattern matching model(s) in the target image buffer.

Syntax

```
void MpatFind(
    AIL_ID ContextPatId,      //in
    AIL_ID TargetImageBufId, //in
    AIL_ID ResultPatId       //out
)
```

Description

This function searches for the model(s) defined in the specified Pattern Matching context and writes the results in the provided result buffer. Note, the Pattern Matching context must be preprocessed (using MpatPreprocess()) before calling this function.

Use MpatControl() to configure the find operation before calling this function.

When there is more than one model, select the appropriate search mode using M_SEARCH_MODE. When the search mode is set to M_FIND_BEST_MODELS, the search settings of the first model apply for the search of all models; otherwise, when using M_FIND_ALL_MODELS, the find operation searches for all models using their own search settings. When using M_FIND_BEST_MODELS, all models must be of the same size.

To specify the maximum number of occurrences for which to look in the target image, use MpatControl() with M_NUMBER. Note that this control type can also be used to temporarily ignore a model during a find operation, by setting M_NUMBER to zero. When using the M_FIND_BEST_MODELS search mode, if M_NUMBER is set to the zero for the first model, the subsequent model that does not have its M_NUMBER set to zero will be used instead.

When the model's reference position is expected to be found in a certain region, you can specify this region using MpatControl() with M_SEARCH_OFFSET_X, M_SEARCH_OFFSET_Y, M_SEARCH_SIZE_X, and M_SEARCH_SIZE_Y. Note that, since you are setting the area in which to find the model's reference position, the search region can be smaller than the model. An alternative method to limiting your search region is to use a rectangular region of interest (ROI), set using MbufSetRegion(). This might cause an increase in processing time, but it is more flexible than setting a search region using MpatControl(); this is because you can define a search region in world units and set it at angle. If using an ROI, the values of M_SEARCH_OFFSET... and M_SEARCH_SIZE... are ignored and the ROI settings take precedence.

Do not use a child buffer to restrict the search region. Using a child buffer could result in edge effects from overscan pixels or incomplete neighborhoods. If the search region is as small as or smaller than the model, using a child buffer could produce invalid results.

To reset the search region size to the full image when using the control types to define the region, set M_SEARCH_OFFSET_X and M_SEARCH_OFFSET_Y to zero (0), and M_SEARCH_SIZE_X and M_SEARCH_SIZE_Y to M_ALL.

This function assumes that the model was extracted from a source image with the same scaling as the target image. In addition, if angular search is disabled (M_SEARCH_ANGLE_MODE is set to M_DISABLE), the model's source image and the target image must have the same orientation (with approximately 5°

tolerance). To perform an angular search, use MpatControl() to enable the angular search and then use M_SEARCH_ANGLE... to set the nominal search angle and the angular search range of the model.

This function returns results in decreasing match-score order. This means that the most-likely occurrence is always returned first.

If a correlation has a match score greater than or equal to the certainty level (MpatControl() with M_CERTAINTY), it is automatically considered an occurrence (default 80%). The remaining occurrences will be the best of those greater than or equal to the acceptance level (MpatControl() with M_ACCEPTANCE).

Results are written to the specified result buffer. Use the MpatGetResult() to read the results.

Parameters

ContextPatId

Specifies the Pattern Matching context identifier to use when finding the search model in the target image.

TargetImageBufId

Specifies the identifier of the target image. Note that this function only supports 8-bit unsigned grayscale images.

This image buffer can only have a rectangular region of interest (ROI) defined (with or without rotation). Besides defining the region in which to search, the angle of the ROI is used as the nominal search angle. In addition, the region must be defined in vector format from a 2D graphics list (M_VECTOR or M_VECTOR_AND_RASTER). An error will be generated if the ROI is only in raster format (M_RASTER) or a non-rectangular shape. An error will be generated if M_SEARCH_ANGLE is set to a value other than 0.0 when the image buffer has an ROI.

ResultPatId

Specifies the identifier of the pattern matching result buffer in which to store results. The result buffer must have been previously allocated using MpatAllocResult().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailpat11.lib.
DLL	Requires ail11.dll; ailpat11.dll.

MpatFree

Synopsis

Free a Pattern Matching context or a result buffer.

Syntax

```
void MpatFree(
    AIL_ID ContextOrResultPatId //in
)
```

Description

This function deletes the specified Pattern Matching context (and all its models) or result buffer identifier, and releases any memory associated with it. Note that, to delete individual models in the context, use MpatDefine().

All Pattern Matching contexts or result buffers allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

ContextOrResultPatId

Specifies the identifier of the Pattern Matching context or result buffer to free. These must have been successfully allocated (with MpatAlloc() or MpatAllocResult()) prior to calling this function.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailpat11.lib.
DLL	Requires ail11.dll; ailpat11.dll.

MpatGetResult

Synopsis

Get the specified type of result(s) from a Pattern Matching result buffer.

Syntax

```
void MpatGetResult(
    AIL_ID ResultPatId,   //in
    AIL_INT Index,        //in
    AIL_INT64 ResultType, //in
    void *ResultArrayPtr  //out
)
```

Description

This function retrieves the result(s) of the specified type from a specified Pattern Matching result buffer. Results are only available after calling MpatFind().

If your target image was associated with a camera calibration context, positional and dimensional results are, by default, returned with respect to the relative coordinate system of the image. Otherwise, these results are returned in pixels, relative to the top-left pixel in the target image.

> If your target image was associated with a camera calibration context but you want to retrieve positional and dimensional results in pixel units, use MpatControl() with the M_RESULT_OUTPUT_UNITS control type set to M_PIXEL. However, note that if you set M_RESULT_OUTPUT_UNITS to M_WORLD without having specified a calibrated target image in which to calculate the results, **MpatGetResult**() will generate an error.

Parameters

ResultPatId

Specifies the identifier of the Pattern Matching result buffer from which to retrieve results.

Index

Specifies where to get results. This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying where to get results																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ALL for result types that apply to model occurrences. Same as M_GENERAL for result types that apply to the entire result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Retrieves the results of the specified type for all model occurrences found.	c		e	f	g	a		i		j	k	l	m	n	b

M_GENERAL	Retrieves a result relating to the entire result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_NUMBER	Specifies the occurrence index.	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result to retrieve for each model occurrence.

See the Parameter associations section for possible values that can be specified.

ResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address of the one-dimensional array in which to write the specified results. The array must be big enough to hold the number of results indicated by M_NUMBER.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ResultType** parameter and possible values returned to the **ResultArrayPtr** parameter.

- For general results
- For specifying the results to retrieve

When retrieving a general result, the ResultType parameter can be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For general results																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																

M_CONTEXT_ID +	Retrieves the identifier of the Pattern Matching context used by MpatFind() to obtain the results in the result buffer. Note that, if the Pattern Matching context has already been freed, using MpatFree(), the returned Pattern Matching context identifier might no longer be valid.														
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER +	Retrieves the number of occurrences found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

To specify the type of result to retrieve for each model occurrence, the ResultType parameter must be set to one of the following.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the results to retrieve																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_ANGLE +	Retrieves the angle of the occurrence (if found using MpatFind()), relative to the output coordinate system specified using MpatControl() with M_RESULT_OUTPUT_UNITS. An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_INDEX +	Retrieves the index of the model that was found.	c		e	f	g	a		i		j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_PIXELS +	Retrieves the number of pixels in the model used to compute the correlation function. Note that to retrieve this result type, the sums must be saved using MpatControl() with M_SAVE_SUMS set to M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_X +	Retrieves the X-coordinate of the occurrence. This is the X-coordinate of the model's reference position transformed at the occurrence (MpatControl() with M_REFERENCE_X).	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_POSITION_Y +	Retrieves the Y-coordinate of the occurrence. This is the Y-coordinate of the model's reference position transformed at the occurrence (MpatControl() with M_REFERENCE_Y.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SCORE +	Retrieves the match score of the occurrence, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SUM_I +	Retrieves the sum of the values of the pixels in the image over the N pixels corresponding to the model occurrence. $r = \sum_{N}^{i=1} I_i$ Note that to retrieve this result type, the sums must be saved using MpatControl() with M_SAVE_SUMS set to M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b

	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SUM_II +		Retrieves the sum of the squares of the values of the pixels in the image over the N pixels corresponding to the model occurrence. $r = \sum_{N}^{i=1} I_i^2$ Note that to retrieve this result type, the sums must be saved using MpatControl() with M_SAVE_SUMS set to M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SUM_IM +		Retrieves the sum of the products of the values of the pixels in the image with the pixels in the model over the N pixels corresponding to the model occurrence. $r = \sum_{N}^{i=1} I_i M_i$ Note that to retrieve this result type, the sums must be saved using MpatControl() with M_SAVE_SUMS set to M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SUM_M +		Retrieves the sum of the values of the pixels in the model over the N pixels of the model. $r = \sum_{N}^{i=1} M_i$ Note that to retrieve this result type, the sums must be saved using MpatControl() with M_SAVE_SUMS set to M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_SUM_MM +	Retrieves the sum of the squares of the values of the pixels in the model over the N pixels of the model. $$r = \sum_{N}^{i=1} M_i^2$$ Note that to retrieve this result type, the sums must be saved using MpatControl() with M_SAVE_SUMS set to M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For general results, For specifying the results to retrieve.

You can add the following value to the above-mentioned values to specify whether a result is available.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether results are available																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_AVAILABLE +	Retrieves whether the requested result type is available for retrieval.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the requested result type is not available.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the requested result type is available.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For specifying the results to retrieve.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																
ResultType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

ResultArrayPtr - Possible values returned																
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For general results, For specifying the results to retrieve; and for the following value: M_AVAILABLE.

You can add one of the following values to the above-mentioned values to cast the requested results to the required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]* **|** *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested results to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]* **|** *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b

	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]* **|** *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64		Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]* **|** *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailpat11.lib.
DLL	Requires ail11.dll; ailpat11.dll.

MpatInquire

Synopsis

Inquire about a setting of a Pattern Matching context, one or all of the search models, or a Pattern Matching result buffer.

Syntax

```
AIL_INT MpatInquire(
    AIL_ID ContextOrResultPatId, //in
    AIL_INT Index,               //in
    AIL_INT64 InquireType,       //in
    void *UserVarPtr             //out
)
```

Description

This function inquires information about a specified Pattern Matching context, one or all of the search models contained therein, or a specified Pattern Matching result buffer. Most inquire type settings can be controlled using MpatControl(). Note that Pattern Matching result buffer values can be obtained with MpatGetResult().

The position (M_ORIGINAL_X and M_ORIGINAL_Y) can be directly compared with search results (by MpatFind()), to calculate the displacement of target images relative to the model's source image.

Parameters

ContextOrResultPatId

Specifies the Pattern Matching context identifier or Pattern Matching result buffer identifier about which to inquire information. The Pattern Matching context or result buffer must have been previously allocated on the required system using MpatAlloc() or MpatAllocResult(), respectively.

Index

Specifies that information will be inquired about the Pattern Matching context, an individual model, or a Pattern Matching result buffer. This parameter should be set to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the index of a model or result buffer																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value. If a Pattern Matching context is specified, this parameter inquires information about model index 0. If a Pattern Matching result buffer is specified, same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONTEXT	Inquires information about a Pattern Matching context, if one is specified.	c		e	f	g	a		i		j	k	l	m	n	b

M_GENERAL	Inquires information about a Pattern Matching result buffer, if one is specified.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Inquires information about a model, if a Pattern Matching context is specified. Set the Index parameter to the index of the required model.	c		e	f	g	a		i		j	k	l	m	n	b

InquireType

Specifies the parameter about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the requested information. Since the **MpatInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring about the system on which the model or result buffer is allocated
- For inquiring about the general settings of the Pattern Matching context
- For inquiring about an individual model in a Pattern Matching context
- For inquiring about the advanced settings
- For inquiring about the result buffer

For the following inquire type, the ContextOrResultPatId can specify a Pattern Matching context or result buffer and the Index parameter must be set to M_CONTEXT or M_GENERAL, respectively.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the system on which the model or result buffer is allocated																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																	
M_OWNER_SYSTEM +	Inquires the system on which the model or result buffer is allocated. SET SET																

UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, which you have allocated using the MsysAlloc() function.	c		e	f	g	a		i		j	k	l	m	n	b

The following inquire types and inquire values allows you to inquire about the settings for a Pattern Matching context. For inquiring about general context settings, the Index parameter must be set to M_CONTEXT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the general settings of the Pattern Matching context																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_CONTEXT_TYPE +	Inquires the type of context used.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_NORMALIZED	Specifies that all search models within the Pattern Matching context should be normalized.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_MODELS +	Inquires the number of models in the context.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of models.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESSED +	Inquires whether the Pattern Matching context is preprocessed. The context must be preprocessed (using MpatPreprocess()) before calling MpatFind(). After certain settings of the Pattern Matching context are changed with MpatControl(), or after a model is added or removed with MpatDefine(), this inquire type will indicate that the context is no longer in its preprocessed state.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

M_FALSE	Specifies the context is not preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies the context is preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_MODE +	Inquires the search mode to use when performing an MpatFind() operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_FIND_ALL_MODELS.	c		e	f	g	a		i		j	k	l	m	n	b
M_FIND_ALL_MODELS	Specifies to search for all the specified models in the target image using their own search settings. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FIND_BEST_MODELS	Specifies to search for the best occurrences of any model in the target image, using the search settings from the first model in the Pattern Matching context. INFO	c		e	f	g	a		i		j	k	l	m	n	b

The following inquire types and inquire values allow you to inquire about the settings for an individual model in a Pattern Matching context. For inquiring about the settings of one model, the Index parameter must be set to the model index of a specific model in the Pattern Matching context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about an individual model in a Pattern Matching context																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_ACCEPTANCE +	Inquires the acceptance level. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the acceptance level, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCURACY +	Inquires the required positional accuracy. Note that the search speed slightly affects the positional accuracy. To inquire the search speed, use M_SPEED. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_HIGH	Specifies a high accuracy (typically ± 0.05 pixels).	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOW	Specifies a low accuracy (typically ± 0.20 pixels). INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEDIUM	Specifies a medium accuracy (typically ± 0.10 pixels).	c		e	f	g	a		i		j	k	l	m	n	b
M_ALLOC_OFFSET_X +		Inquires the X-offset of model's top-left corner relative to the top-left corner of the model's source image.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-offset, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALLOC_OFFSET_Y +		Inquires the Y-offset of model's top-left corner relative to the top-left corner of the model's source image.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-offset, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALLOC_SIZE_BIT +		Inquires the depth of the model image. Note that the model image is a single-band image.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	1	Specifies that the data depth is 1 bit.	c		e	f	g	a		i		j	k	l	m	n	b
	8	Specifies that the data depth is 8 bits.	c		e	f	g	a		i		j	k	l	m	n	b

	16	Specifies that the data depth is 16 bits.	c		e	f	g	a		i		j	k	l	m	n	b
	32	Specifies that the data depth is 32 bits.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALLOC_SIZE_X +		Inquires the model's width.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the width, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALLOC_SIZE_Y +		Inquires the model's height.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_CERTAINTY +		Inquires the certainty level. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the certainty level, as a percentage.	c		e	f	g	a		i		j	k	l	m	n	b
M_HAS_DONT_CARE_MASK +		Inquires whether the model has an M_DONT_CARE mask applied to it.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that an M_DONT_CARE mask has not been applied to the model.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that an M_DONT_CARE mask has been applied to the model.	c		e	f	g	a		i		j	k	l	m	n	b

M_MODEL_TYPE +		Inquires the type of model used.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_AUTO_MODEL +	Defines one or more models from the most-suitable distinct areas found, of the specified dimensions, in the model's source image.	c		e	f	g	a		i		j	k	l	m	n	b
	M_REGULAR_MODEL +	Defines a regular search model, using data from the specified area of the model's source image.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER +		Inquires the maximum expected number of occurrences of a search model to find in a target image. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies to look for all matches that are greater than or equal to the acceptance level.	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the maximum number of occurrences for which to look in the target image. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_ORIGINAL_X +		Inquires the X-offset of the model's reference position relative to the top-left corner of the model's source image (takes into account the M_REFERENCE_X setting).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-offset.	c		e	f	g	a		i		j	k	l	m	n	b

M_ORIGINAL_Y +		Inquires the Y-offset of the model's reference position relative to the top-left corner of the model's source image (takes into account the M_REFERENCE_Y setting).	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-offset.	c		e	f	g	a		i		j	k	l	m	n	b
M_REFERENCE_X +		Inquires the x-offset of the origin of the model's reference position relative to the model origin. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-offset of the model's reference position relative to the model origin, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_REFERENCE_Y +		Inquires the y-offset of the origin of the model's reference position relative to the model origin. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-offset of the model's reference position relative to the model origin, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_ROTATED_MODEL_MINIMUM_SCORE +		Inquires whether to use angle refinement mode as well as sets the score to use for this mode. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies to disable the angle refinement mode.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 100.0	Specifies the minimum score.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE +		Inquires whether search for occurrences of the model at the nominal search angle. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 360.0	Specifies the nominal search angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_ACCURACY +		Inquires the required precision of the nominal angle. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies that the angle accuracy equals the tolerance.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 < *Value* <= 180.0	Specifies the angle accuracy, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_DELTA_NEG +		Inquires the lower limit for the angular range, relative to the nominal search angle. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 <= *Value* <= 180.0	Specifies the angle in a clockwise rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_DELTA_POS +		Inquires the upper limit for the angular range, relative to the nominal search angle. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	0.0 <= *Value* <= 180.0	Specifies the angle in a counter-clockwise rotation, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_INTERPOLATION_MODE +		Inquires the type of interpolation to use when rotating the model to perform the search at an angle. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_BICUBIC	Specifies bicubic interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_BILINEAR	Specifies bilinear interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_NEAREST_NEIGHBOR	Specifies nearest neighbor interpolation. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_MODE +		Inquires whether to perform calculations specific to angular-range search strategies. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies to disable the angular search mode.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to enable the angular search mode.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_TOLERANCE +		Inquires the full range of degrees within which the pattern in the target image can be rotated from a model at a specific angle and still meet the acceptance level. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	M_AUTO	Specifies that the tolerance will be automatically determined based on an analysis of the model.	c		e	f	g	a		i		j	k	l	m	n	b
	0.0 < *Value* <= 180.0	Specifies the step angle, in degrees.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_OFFSET_X +		Inquires the X-offset of the specified model's search region. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the X-coordinate, relative to the pixel coordinate system, of where to start the search. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_OFFSET_Y +		Inquires the Y-offset of the specified model's search region. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the Y-coordinate, relative to the pixel coordinate system, of where to start the search. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_SIZE_X +		Inquires the width of the search region. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies to use the width of the full image.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the width, in pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_SIZE_Y +		Inquires the height of the search region. SET	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies to use the height of the full image.	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the height, in pixels. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SPEED +		Inquires the required search speed when using MpatFind(). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_HIGH	Specifies a high search speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOW	Specifies a low search speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEDIUM	Specifies a medium search speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_VERY_HIGH	Specifies a very high search speed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_VERY_LOW	Specifies a very low search speed.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for [when InquireType=M_MODEL_TYPE: M_AUTO_MODEL; M_REGULAR_MODEL].

The following value might be combined with the above-mentioned values to determine whether circular overscan data was extracted with the model.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

Returns whether circular overscan data was extracted with the model																
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CIRCULAR_OVERSCAN	Specifies that the model is extracted from the source image with the circular overscan data. INFO	c		e	f	g	a		i		j	k	l	m	n	b

The following inquire types and inquire values are for advanced users of the Pattern Matching module. If the default settings do not satisfy the requirements of your application, these inquire types and inquire values can be used to fine tune the model's search settings. The default settings typically provide the best results for most search operations. To gain a clearer understanding of how the search algorithm works,

see the Pattern matching algorithm (for advanced users) section of Chapter 7: Pattern matching.

For these inquire types, the Index parameter must be set to the model index of a specific model in the Pattern Matching context.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the advanced settings																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_COARSE_SEARCH_ACCEPTANCE +		Inquires the coarse search acceptance threshold level to use for rejecting candidate model peaks at low resolution levels. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies to determine the coarse search acceptance threshold level automatically.	c		e	f	g	a		i		j	k	l	m	n	b
	1.0 <= *Value* <= 100.0	Specifies the coarse search acceptance threshold level.	c		e	f	g	a		i		j	k	l	m	n	b
M_EXTRA_CANDIDATES +		Inquires the number of extra candidates to consider. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 0	Specifies the number of extra candidates. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_FAST_FIND +		Inquires whether to use fast peak finding. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies not to use fast peak finding. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies to use fast peak finding. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_FIRST_LEVEL +		Inquires the resolution level for the initial stage (lowest resolution) of the search. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= *MaxLevel*	Specifies the first level, where MaxLevel is the maximum possible level. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_LAST_LEVEL +		Inquires the resolution level for the final stage (highest resolution) of the search. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= *MaxLevel*	Specifies the last level, where MaxLevel is the maximum possible level. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_INITIAL_PEAKS +		Inquires the maximum number of returned peaks after an initial search at the first level for occurrences of multiple models, or a single model with circular overscan data (that is, M_AUTO_MODEL or M_REGULAR_MODEL with M_CIRCULAR_OVERSCAN). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies to limit the returned number of peaks for optimal performance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies all occurrences that pass the model acceptance threshold should be returned. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_SEPARATION_X +		Inquires the minimum separation (in X) between two occurrences of the same model in order for them to be considered distinct. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

	Value	Specifies the minimum separation as a percentage of the model's width. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_SEPARATION_Y +		Inquires the minimum separation (in Y) between two occurrences of the same model in order for them to be considered distinct. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value	Specifies the minimum separation as a percentage of the model's height. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_MODEL_MAX_LEVEL +		Inquires the maximum search level possible for the model.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	Value >= 7	Specifies the maximum search level possible for the model.	c		e	f	g	a		i		j	k	l	m	n	b
M_MODEL_STEP +		Inquires whether all or every second pixel in the model is used in the correlation during the high resolution stage of the search. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies to automatically establish the model step.	c		e	f	g	a		i		j	k	l	m	n	b
	1	Specifies to use all model pixels.	c		e	f	g	a		i		j	k	l	m	n	b
	2	Specifies to use every second model pixel (in both the X and Y directions). INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_FIRST_LEVEL +		Inquires the default resolution level for the initial stage (lowest resolution) of the search.	c		e	f	g	a		i		j	k	l	m	n	b

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= *MaxLevel*	Specifies the lowest resolution level, where *MaxLevel* is the maximum level. To establish the maximum level, use M_MODEL_MAX_LEVEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_LAST_LEVEL +		Inquires the highest level for the final stage (highest resolution) of the search.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* <= *MaxLevel*	Specifies the lowest resolution level, where MaxLevel is the maximum level. To establish the maximum level, use M_MODEL_MAX_LEVEL.	c		e	f	g	a		i		j	k	l	m	n	b
M_SEARCH_ANGLE_ACCURACY_MODE +		Inquires the method for scanning the specified angular range for multiple occurrences, or a single occurrence with circular overscan data (that is, M_AUTO_MODEL or M_REGULAR_MODEL with M_CIRCULAR_OVERSCAN). SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies to return the closest occurrence to the approximate location within the specified angular range, for optimal performance.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL	Specifies that all possible occurrences that are found within the range of the accuracy specified with M_SEARCH_ANGLE_ACCURACY will be scanned, and the best will be returned. INFO	c		e	f	g	a		i		j	k	l	m	n	b

For a result buffer identifier, the InquireType parameter can be set to one of the values below. For inquiring about the settings of a Pattern Matching result buffer, the Index parameter must be set to M_GENERAL.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the result buffer																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_RESULT_OUTPUT_UNITS +	Inquires whether to return results in pixels or world units. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, it specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE_SUMS +	Inquires whether to save the values of the sums used to compute the normalized correlation result for each model occurrence. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that the sums are not saved.	c		e	f	g	a		i		j	k	l	m	n	b
M_ENABLE	Specifies that the sums are saved.	c		e	f	g	a		i		j	k	l	m	n	b
M_TARGET_CACHING +	Inquires whether the pyramidal representation of the target buffer is kept in the result buffer or generated each time MpatFind() is called. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_ENABLE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies that the pyramidal representation of the target buffer is generated each time MpatFind() is called.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ENABLE	Specifies that the pyramidal representation of the target buffer is kept in the result buffer. INFO	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- For inquiring about the system on which the model or result buffer is allocated
- For inquiring about the general settings of the Pattern Matching context
- For inquiring about an individual model in a Pattern Matching context
- For inquiring about the advanced settings
- For inquiring about the result buffer

the values listed in all the parameter association tables.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For casting information to the required data type.																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																	
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.																
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested information to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b

M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT*. If the requested information does not fit into an *AIL_INT*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailpat11.lib.
DLL	Requires ail11.dll; ailpat11.dll.

MpatPreprocess

Synopsis

Preprocess a Pattern Matching context.

Syntax

```
void MpatPreprocess(
    AIL_ID ContextPatId,     //in
    AIL_INT64 ControlFlag,   //in
    AIL_ID TypicalImageBufId //in
)
```

Description

This function preprocesses the specified Pattern Matching context. It trains the Aurora Imaging Library application to search for one or more models in the most efficient manner (optionally within a specified typical image). The procedure is potentially quite lengthy (up to several seconds).

Call this function after at least one search model is added to the context and all search settings have been adjusted. When you save, the model's preprocessing changes are not stored with the model. Upon restoration, the model needs to be preprocessed.

Note that if some of the model's search settings are changed after a call to **MpatPreprocess**(), the model must be preprocessed again. To inquire if your model is in a preprocessed state, use MpatInquire() with M_PREPROCESSED.

Parameters

ContextPatId

Specifies the identifier of the Pattern Matching context to preprocess.

ControlFlag

Reserved for future expansion. Set this parameter to **M_DEFAULT**.

TypicalImageBufId

Specifies the identifier of a typical target image. The specified typical image will be used to refine and adapt the model to search on this typical background. You should only specify an image buffer if the models will always appear on such a background; otherwise, set this parameter to **M_NULL**.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailpat11.lib.
DLL	Requires ail11.dll; ailpat11.dll.

MpatMask

Synopsis

Mask areas of a model in a Pattern Matching context.

Syntax

```
void MpatMask(
    AIL_ID ContextPatId,  //out
    AIL_INT Index,        //in
    AIL_ID MaskBufferId,  //in
    AIL_INT64 MaskType,   //in
    AIL_INT64 ControlFlag //in
)
```

Description

This function allows you to apply a "don't care" mask to a model in a Pattern Matching context. Essentially, the mask sets pixels in the specified model to a "don't care state. The "don't care" pixels of the model will not be considered when searching for occurrences of the model in the target image (using MpatFind()). If a pixel in the mask image is a non-zero value, the corresponding pixel in the model is set to a "don't care" state. Note that there must be at least one model to the context before calling this function in the mask image; otherwise an error will occur.

Note, each time this function is called, a new set of "don't care" pixels is assigned to the specified model, superseding any existing set. Therefore, multiple calls to this function do not have a cumulative effect. Also, whenever the "don't care" pixels in a model change, the effect of any preprocessing is undone. Therefore, if the "don't care" pixels are changed, call MpatPreprocess() before searching for the model.

> This function does not support an M_REGULAR_MODEL + M_CIRCULAR_OVERSCAN type of model, except if M_SEARCH_ANGLE_MODE is set to M_DISABLE.

Parameters

ContextPatId

Specifies the identifier of the Pattern Matching context containing the model to mask.

Index

Specifies the individual model on which to apply the mask.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying a model																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b

Value >= 0	Specifies the index of the model for which the mask is being created.	c		e	f	g	a		i		j	k	l	m	n	b

MaskBufferId

Specifies the identifier of the image buffer to use as a mask. Non-zero-pixel values identify the "don't care" pixels in the model.

This buffer must be a 1-band, 8-bit, unsigned image buffer. The buffer must be of the same size as the model. The buffer can be calibrated; however, the mask is applied on a pixel basis.

This image buffer must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

Note that you can set this parameter to **M_NULL** to remove the mask.

MaskType

Specifies which type of mask to employ.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the value of the don't care pixels																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DONT_CARE	Specifies to apply (or remove) a don't care mask. The features present in the masked area (non-zero pixels) are ignored during the search. Features present in this area of the occurrence will not affect the scores.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailpat11.lib.
DLL	Requires ail11.dll; ailpat11.dll.

MpatRestore

Synopsis

Restore a Pattern Matching context from disk.

Syntax

```
AIL_ID MpatRestore(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SysId,                //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID *ContextPatIdPtr      //out
)
```

Description

This function restores a Pattern Matching context that was previously saved to a file, using MpatSave(). A restored Pattern Matching context is not preprocessed, therefore you must call MpatPreprocess() before performing a search with MpatFind().

When the restored Pattern Matching context is no longer required, release it using MpatFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the restored Pattern Matching context's lifetime and you must not manually free it.

Parameters

FileName

Specifies the name and path of the file from which to restore the model. The function handles (internally) the opening and closing of the file.

This parameter can be set to one of the following:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Open** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, Pattern Matching context files have an MPAT extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the system on which to restore the model.

This parameter should be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, which you have allocated using the MsysAlloc() function.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

ContextPatIdPtr

Specifies the address of the variable in which to write the Pattern Matching context identifier or specifies the data type that the function should use to return the Pattern Matching context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated Pattern Matching context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated Pattern Matching context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_PAT_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the Pattern Matching context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the restored Pattern Matching context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the Pattern Matching context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_PAT_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailpat11.lib.
DLL	Requires ail11.dll; ailpat11.dll.

MpatSave

Synopsis

Save a Pattern Matching context to disk.

Syntax

```
void MpatSave(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID ContextPatId,         //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function saves all the information about the previously allocated Pattern Matching context to disk, including all of the Pattern Matching context's current search parameters values. Later, this information can be reloaded, using MpatRestore() or MpatStream().

All of the Pattern Matching context settings that are enabled or were in effect are saved. A loaded or restored Pattern Matching context is not preprocessed.

Parameters

FileName

Specifies the name and path of the file in which to save the model. It is recommended that you use the MPAT file extension for easier use with other Zebra Aurora software products. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten.

This parameter can be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens the **File Save As** dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"). Typically, Pattern Matching context files have an MPAT extension. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ContextPatId

Specifies the identifier of the Pattern Matching context to save.

ControlFlag

Reserved for future expansion. This parameter must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailpat11.lib.
DLL	Requires ail11.dll; ailpat11.dll.

MpatStream

Synopsis

Load, restore, or save a Pattern Matching context from/to a file or a memory stream.

Syntax

```
void MpatStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SysId,                  //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *ContextPatIdPtr,       //in-out
    AIL_INT *SizeByteVarPtr        //out
)
```

Description

This function can load, restore, or save a Pattern Matching context from/to a file or memory stream.

To inquire the number of bytes necessary to save a Pattern Matching context to memory stream, you should first call this function (**MpatStream**()) with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. In addition, any file saved using this function is equivalent to a file saved with MpatSave().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file. All information about the previously allocated Pattern Matching context is saved, including all of the Pattern Matching context's current search settings. However, any effects of preprocessing are not saved.

All of the Pattern Matching context settings that were in effect when the Pattern Matching context was saved will be restored. A loaded or restored Pattern Matching context is not preprocessed, therefore you must call MpatPreprocess() before performing a search with MpatFind().

Using **MpatStream**(), you can choose to save a backwards-compatible version of the Pattern Matching context, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore Pattern Matching contexts saved using any Aurora Imaging Library version with which this function was available. Settings that do not exist in the lower version will be filled with default values when the Pattern Matching context is loaded or restored.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the system on which to restore the Pattern Matching context.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform on the Pattern Matching context.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the Pattern Matching context. This parameter must be set to one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of stream																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the Pattern Matching context.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextPatIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the Pattern Matching context.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the Pattern Matching context, in bytes. If the size is not required, you can set this parameter to **M_NULL**.

> Note that the size of a Pattern Matching context will vary depending on the Aurora Imaging Library version specified.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **ContextPatIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.																	
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
MemPtrOrFileName - Possible values passed or returned																
SysId																
Version																
ContextPatIdPtr - Possible values passed or returned																
M_INQUIRE_SIZE_BYTE	Inquires the number of bytes required to save a Pattern Matching context to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.															
MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the Pattern Matching context must be compatible when saved. The size of a Pattern Matching context might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b

	M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
ContextPatIdPtr		Specifies the address of the variable containing the identifier of the Pattern Matching context.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD		Loads the content of a specified file or memory stream into a previously allocated Pattern Matching context.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName		Specifies the file or memory stream from which to load the context.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Pattern matching models typically have an MPAT file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b

		MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId		This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
	Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
	ContextPatIdPtr		Specifies the address of the variable containing the identifier of the Pattern Matching context. All previous settings are overwritten with those of the loaded context, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the context is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE			Restores a Pattern Matching context from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtrOrFileName		Specifies the file or memory stream from which to restore the context.	c		e	f	g	a		i		j	k	l	m	n	b
		M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b

	AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. Pattern Matching contexts typically have an MPAT file extension. The function handles (internally) the opening and closing of the file. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId		Specifies the system on which to allocate the Pattern Matching context.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
	System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
Version		This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextPatIdPtr		Specifies the address of the variable in which to write the identifier of the Pattern Matching context. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAVE		Saves a Pattern Matching context to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b

	MemPtrOrFileName	Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the file (for example, "*C:\mydirectory\myfile*"), when the StreamType parameter is set to M_FILE. For easier use with other Zebra Aurora software products, when saving a code context to a file, use the MPAT file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\myfile*").	c		e	f	g	a		i		j	k	l	m	n	b
	MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
	SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b

	Version	Specifies the Aurora Imaging Library version with which the Pattern Matching context must be compatible when saved. Note that the Pattern Matching context can only be up to one major release older than the current version.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP4	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 4.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP5	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 5.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP6	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 6.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
	ContextPatIdPtr	Specifies the address of the variable containing the identifier of the pattern matching model to save.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; ailpat11.lib.
DLL	Requires ail11.dll; ailpat11.dll.